

Οδηγός Ζωντανής Παρουσίασης 10 Λεπτών

Χρησιμοποίησε αυτό το αρχείο στο tablet την ώρα της παρουσίασης. Είναι γραμμένο σαν διαδρομή: ξεκινάς από το πρώτο αρχείο, λες το βασικό νόημα, δείχνεις τις σημαντικές γραμμές, και μετά πας στο επόμενο.

0. Αν Κολλήσεις

Πες αυτό:

Το project είναι μια εφαρμογή PHP και MariaDB για appointment lists. Οι χρήστες μπορούν να κάνουν register, login, να ψάχνουν ranked candidate entries και να παρακολουθούν candidates. Οι admins μπορούν να διαχειρίζονται specialties, appointment lists, candidates και candidate-list entries. Η βάση έχει 7 συνδεδεμένους πίνακες, όλη η πρόσβαση στη βάση γίνεται με PDO, οι κωδικοί αποθηκεύονται hashed, οι protected σελίδες χρησιμοποιούν PHP sessions, και το API folder δίνει JSON endpoints για Postman testing.

Demo accounts:

```
admin@example.com / Password123!  
writer@example.com / Password123!  
viewer@example.com / Password123!
```

1. Αρχή: Entry Point Του Project

Άνοιξε: `index.php`

Σημαντικές γραμμές:

- `index.php:1` ξεκινά PHP.
- `index.php:2` ξεκινά session.
- `index.php:4-5` φορτώνει database και helper functions.
- `index.php:7-14` φορτώνει live στατιστικά από τη βάση.
- `index.php:36-47` αλλάζει navigation ανάλογα με login και role.
- `index.php:59-71` δείχνει τις 3 βασικές υπηρεσίες: search, tracking, administration.

Κώδικας να δείξεις:

```
session_start();  
  
require_once __DIR__ . '/includes/db.php';  
require_once __DIR__ . '/includes/functions.php';
```

Τι να πεις:

Η homepage είναι το δημόσιο entry point. Ξεκινά session, φορτώνει τα κοινά database και helper αρχεία, και μετά διαβάζει πραγματικά counts από τη βάση. Το navigation αλλάζει ανάλογα με το αν ο χρήστης είναι logged in και αν είναι admin.

Μετάβαση:

Από εδώ θα δείξω το shared database connection, γιατί όλες οι σελίδες και όλα τα API endpoints βασίζονται σε αυτό.

2. Database Connection

Άνοιξε: `includes/db.php`

Σημαντικές γραμμές:

- `includes/db.php:4-8` ορίζει database settings.
- `includes/db.php:10` ορίζει τη shared function `db()`.
- `includes/db.php:12-16` ξαναχρησιμοποιεί το ίδιο PDO connection.
- `includes/db.php:23-28` φτιάχνει το MySQL DSN με `utf8mb4`.
- `includes/db.php:30-38` δημιουργεί το PDO object.
- `includes/db.php:35-36` ενεργοποιεί exceptions και associative array results.

Κώδικας να δείξεις:

```
$pdo = new PDO(
    $dsn,
    DB_USER,
    DB_PASS,
    [
        PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
        PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
    ]
);
```

Τι να πεις:

Όλη η πρόσβαση στη βάση περνά από αυτό το helper. Χρησιμοποιεί PDO, ενεργοποιεί exceptions, και επιστρέφει rows σαν associative arrays. Έτσι η συμπεριφορά της βάσης είναι ίδια σε όλες τις σελίδες και στα API endpoints.

Μετάβαση:

Τώρα θα δείξω τα helper functions που προστατεύουν τις σελίδες και κάνουν escape το output.

3. Shared Security Helpers

Άνοιξε: `includes/functions.php`

Σημαντικές γραμμές:

- `includes/functions.php:4-8` κάνει redirect και σταματά την εκτέλεση.

- `includes/functions.php:10-13` ελέγχει αν υπάρχει logged-in user.
- `includes/functions.php:15-20` προστατεύει member pages.
- `includes/functions.php:32-40` διαβάζει role και ελέγχει admin.
- `includes/functions.php:42-50` προστατεύει admin pages.
- `includes/functions.php:52-55` κάνει escape output.
- `includes/functions.php:77-89` καθαρίζει form input.
- `includes/functions.php:92-104` γράφει audit logs.

Κώδικας να δείξεις:

```
function require_admin(): void
{
    require_auth();

    if (!is_admin()) {
        http_response_code(403);
        exit('Forbidden');
    }
}
```

Και:

```
function e(?string $value): string
{
    return htmlspecialchars((string) $value, ENT_QUOTES, 'UTF-8');
}
```

Τι να πεις:

Αυτά τα helpers κρατούν την επαναλαμβανόμενη ασφάλεια σε ένα μέρος. Το `require_auth()` μπλοκάρει anonymous users, το `require_admin()` μπλοκάρει non-admin users, και το `e()` κάνει escape πριν εμφανιστεί κάτι στο HTML.

Μετάβαση:

Αυτά βασίζονται στο session, άρα τώρα θα δείξω registration και login.

4. Registration

Άνοιξε: `auth/register.php`

Σημαντικές γραμμές:

- `auth/register.php:2` ξεκινά session.
- `auth/register.php:4-5` φορτώνει database και helpers.
- `auth/register.php:15-19` διαβάζει input από τη φόρμα.
- `auth/register.php:21-41` κάνει validation για username, email, password και confirmation.
- `auth/register.php:47-57` ελέγχει duplicate email και username.

- `auth/register.php:60-68` εισάγει τον νέο χρήστη.
- `auth/register.php:66` κάνει hash τον κωδικό.
- `auth/register.php:67` βάζει default role `member`.
- `auth/register.php:70` κάνει redirect στο login.

Κώδικας να δείξεις:

```
'password_hash' => password_hash($password, PASSWORD_DEFAULT),  
'role' => 'member',
```

Τι να πεις:

To registration γίνεται server-side. Ελέγχει τα inputs, ψάχνει για duplicate email και username, και αποθηκεύει password hash, όχι plain text password. Οι νέοι χρήστες γίνονται members by default.

Μετάβαση:

Μετά το registration ο χρήστης κάνει login. Εκεί δημιουργείται το session state.

5. Login Και Session State

Άνοιξε: `auth/login.php`

Σημαντικές γραμμές:

- `auth/login.php:2` ξεκινά session.
- `auth/login.php:7-9` κάνει redirect αν ο χρήστης είναι ήδη logged in.
- `auth/login.php:15-16` χειρίζεται POST login request.
- `auth/login.php:22-24` βρίσκει τον user με email.
- `auth/login.php:26` ελέγχει τον password hash.
- `auth/login.php:29-31` αποθηκεύει `user_id`, `role`, `username` στο session.
- `auth/login.php:33` κάνει redirect στο protected dashboard.

Κώδικας να δείξεις:

```
if (!$user || !password_verify($password, $user['password_hash'])) {  
    $errors[] = 'Λανθασμένα στοιχεία σύνδεσης.';  
} else {  
    $_SESSION['user_id'] = (int) $user['id'];  
    $_SESSION['role'] = (string) $user['role'];  
    $_SESSION['username'] = (string) $user['username'];  
}
```

Τι να πεις:

To login χρησιμοποιεί `password_verify()` πάνω στο αποθηκευμένο hash. Αν πετύχει, το session γίνεται η πηγή αλήθειας για το current user id, role και username.

Μετάβαση:

Αφού γίνει authentication, ο χρήστης πηγαίνει στο protected dashboard.

6. Protected User Dashboard

Άνοιξε: `modules/dashboard.php`

Σημαντικές γραμμές:

- `modules/dashboard.php:7` απαιτεί authentication.
- `modules/dashboard.php:11-18` φορτώνει dashboard statistics.
- `modules/dashboard.php:20-29` φορτώνει τα πρόσφατα tracked candidates του current user.
- `modules/dashboard.php:61-64` εμφανίζει username και role από το session.
- `modules/dashboard.php:86-91` δείχνει actions και admin link μόνο όπου πρέπει.

Κώδικας να δείξεις:

```
require_auth();
```

Και:

```
You are signed in as <strong><?= e(current_role()) ?></strong>.
```

Τι να πεις:

Αυτή η σελίδα αποδεικνύει ότι το session δουλεύει. Προστατεύεται με `require_auth()` και μετά χρησιμοποιεί το current session για να εμφανίσει username, role, στατιστικά και προσωπικό tracking.

Μετάβαση:

Το βασικό member feature είναι το search στα appointment-list entries.

7. Search Page

Άνοιξε: `modules/list.php`

Σημαντικές γραμμές:

- `modules/list.php:7` απαιτεί login.
- `modules/list.php:9-11` διαβάζει GET filters.
- `modules/list.php:18-35` χτίζει το βασικό SQL query.
- `modules/list.php:30-33` κάνει join entries, candidates, appointment lists και specialties.
- `modules/list.php:39-50` προσθέτει keyword search.
- `modules/list.php:52-60` προσθέτει status και specialty filters.
- `modules/list.php:64-66` κάνει prepare, execute και fetch.
- `modules/list.php:107-138` εμφανίζει τη filter form.

- `modules/list.php:149-160` εμφανίζει τα result cards.

Κώδικας να δείξεις:

```
FROM candidate_list_entries
INNER JOIN candidates ON candidates.id =
candidate_list_entries.candidate_id
INNER JOIN appointment_lists ON appointment_lists.id =
candidate_list_entries.list_id
INNER JOIN specialties ON specialties.id = appointment_lists.specialty_id
```

Και:

```
$stmt = $pdo->prepare($sql);
$stmt->execute($params);
$entries = $stmt->fetchAll();
```

Τι να πεις:

Η search page είναι protected και χρησιμοποιεί GET filters, άρα το search μπορεί να γίνει bookmark. Το σημαντικό είναι το join: το αποτέλεσμα που βλέπει ο χρήστης έρχεται από τέσσερις related tables, όχι από έναν flat πίνακα.

Μετάβαση:

Οι members μπορούν επίσης να σώζουν candidates στη δική τους watchlist.

8. Member Tracking

Άνοιξε: `modules/tracking/dashboard.php`

Σημαντικές γραμμές:

- `modules/tracking/dashboard.php:7` απαιτεί login.
- `modules/tracking/dashboard.php:13-22` χειρίζεται delete.
- `modules/tracking/dashboard.php:24-29` κάνει validation σε candidate και label.
- `modules/tracking/dashboard.php:30-41` κάνει insert ή update tracked candidate.
- `modules/tracking/dashboard.php:47-58` φορτώνει τα tracked candidates του user.
- `modules/tracking/dashboard.php:102-121` εμφανίζει τη tracking form.
- `modules/tracking/dashboard.php:133-140` εμφανίζει κάθε tracked candidate και remove button.

Κώδικας να δείξεις:

```
INSERT INTO tracked_candidates (user_id, candidate_id, label, is_active)
VALUES (:user_id, :candidate_id, :label, 1)
ON DUPLICATE KEY UPDATE label = VALUES(label), is_active = 1
```

Τι να πεις:

Το tracking είναι member-level CRUD. Το record ανήκει στον current logged-in user. Το unique rule στη βάση εμποδίζει τον ίδιο user να κάνει track τον ίδιο candidate δύο φορές, οπότε ο κώδικας απλώς ενημερώνει το label.

Μετάβαση:

Τώρα θα αλλάξω στο admin role και θα δείξω το admin dashboard.

9. Admin Hub

Άνοιξε: `modules/admin/dashboard.php`

Σημαντικές γραμμές:

- `modules/admin/dashboard.php:7` απαιτεί admin role.
- `modules/admin/dashboard.php:11-16` φορτώνει admin metrics.
- `modules/admin/dashboard.php:18-23` φορτώνει recent audit logs.
- `modules/admin/dashboard.php:48-76` δείχνει links για τα 4 CRUD modules.
- `modules/admin/dashboard.php:102-118` εμφανίζει recent audit records.

Κώδικας να δείξεις:

```
require_admin();
```

Τι να πεις:

Το admin hub προστατεύεται με `require_admin()`, οπότε ένας απλός member δεν μπορεί να μπει ακόμα και αν ξέρει το URL. Από εδώ ο admin διαχειρίζεται τα τέσσερα βασικά entities του project.

Μετάβαση:

Τα τέσσερα CRUD pages έχουν το ίδιο pattern, οπότε θα δείξω ένα αναλυτικά και μετά θα αναφέρω τα υπόλοιπα.

10. CRUD Pattern: Specialties Example

Άνοιξε: `modules/specialties/dashboard.php`

Σημαντικές γραμμές:

- `modules/specialties/dashboard.php:7` απαιτεί admin.
- `modules/specialties/dashboard.php:14-24` χειρίζεται delete.

- `modules/specialties/dashboard.php:26-35` διαβάζει και κάνει validate form data.
- `modules/specialties/dashboard.php:37-58` επιλέγει update ή insert.
- `modules/specialties/dashboard.php:45` γράφει audit log για update.
- `modules/specialties/dashboard.php:54` γράφει audit log για create.
- `modules/specialties/dashboard.php:61-72` φορτώνει record για editing.
- `modules/specialties/dashboard.php:75` φορτώνει όλα τα specialties.
- `modules/specialties/dashboard.php:103-124` εμφανίζει τη φόρμα.
- `modules/specialties/dashboard.php:140-153` εμφανίζει τα table rows.

Κώδικας να δείξεις:

```
if ($id > 0) {  
    $stmt = $pdo->prepare(  
        'UPDATE specialties  
        SET code = :code, name = :name, sector = :sector  
        WHERE id = :id'  
    );  
} else {  
    $stmt = $pdo->prepare(  
        'INSERT INTO specialties (code, name, sector)  
        VALUES (:code, :name, :sector)'  
    );  
}
```

Τι να πεις:

Αυτό είναι το CRUD pattern που χρησιμοποιούν τα admin pages: ελέγχουν admin role, διαβάζουν POST data, κάνουν validation, χρησιμοποιούν prepared statements, γράφουν audit log, βάζουν flash message και κάνουν redirect.

Άλλα CRUD pages:

- `modules/lists/dashboard.php:7` admin guard, `modules/lists/dashboard.php:14-59` CRUD logic, `modules/lists/dashboard.php:78-83` κάνει join lists με specialties.
- `modules/candidates/dashboard.php:7` admin guard, `modules/candidates/dashboard.php:14-62` CRUD logic, `modules/candidates/dashboard.php:35-39` κάνει required fields και email validation.
- `modules/entries/dashboard.php:7` admin guard, `modules/entries/dashboard.php:14-60` CRUD logic, `modules/entries/dashboard.php:81-91` κάνει join entries με candidates και lists.

Μετάβαση:

Αυτές οι σελίδες δουλεύουν καθαρά επειδή η βάση είναι relational.

11. Database Schema

Άνοιξε: `database/schema.sql`

Σημαντικές γραμμές:

- `database/schema.sql:1-2` δημιουργεί και επιλέγει database.
- `database/schema.sql:4-11` δημιουργεί `users`.
- `database/schema.sql:17-23` δημιουργεί `specialties`.
- `database/schema.sql:25-36` δημιουργεί `appointment_lists`.
- `database/schema.sql:32-35` συνδέει appointment lists με specialties.
- `database/schema.sql:38-46` δημιουργεί `candidates`.
- `database/schema.sql:48-65` δημιουργεί `candidate_list_entries`.
- `database/schema.sql:56-63` συνδέει entries με candidates και lists.
- `database/schema.sql:67-83` δημιουργεί `tracked_candidates`.
- `database/schema.sql:74-82` συνδέει tracking με users και candidates.
- `database/schema.sql:85-96` δημιουργεί `audit_logs`.

Tables:

```
users
specialties
appointment_lists
candidates
candidate_list_entries
tracked_candidates
audit_logs
```

Κώδικας να δείξεις:

```
CONSTRAINT fk_entries_candidate
  FOREIGN KEY (candidate_id) REFERENCES candidates(id)
```

και:

```
CONSTRAINT fk_entries_list
  FOREIGN KEY (list_id) REFERENCES appointment_lists(id)
```

Τι να πεις:

Η βάση προστατεύει τις σχέσεις με foreign keys. Ένα candidate-list entry δεν μπορεί να υπάρχει αν δεν δείχνει σε πραγματικό candidate και πραγματικό appointment list.

Μετάβαση:

Τα ίδια δεδομένα είναι διαθέσιμα και σαν JSON μέσα από το API folder.

12. API Bootstrap

Άνοιξε: `api/_bootstrap.php`

Σημαντικές γραμμές:

- `api/_bootstrap.php:4` φορτώνει το database helper.
- `api/_bootstrap.php:6` βάζει JSON content type.
- `api/_bootstrap.php:8-13` ορίζει standard JSON response function.
- `api/_bootstrap.php:15-19` ανοίγει database ή επιστρέφει JSON error.

Κώδικας να δείξεις:

```
header('Content-Type: application/json; charset=utf-8');

function api_response(array $payload, int $status = 200): never
{
    http_response_code($status);
    echo json_encode($payload, JSON_PRETTY_PRINT | JSON_UNESCAPED_UNICODE);
    exit;
}
```

Τι να πεις:

Τα API endpoints μοιράζονται αυτό το bootstrap. Βάζει JSON header, συνδέεται στη βάση και δίνει κοινή μορφή response σε όλα τα endpoints.

Μετάβαση:

To API search endpoint κάνει mirror το browser search page.

13. API Search

Άνοιξε: `api/search/index.php`

Σημαντικές γραμμές:

- `api/search/index.php:4` φορτώνει API bootstrap.
- `api/search/index.php:6-8` διαβάζει filters από query string.
- `api/search/index.php:10-26` χτίζει joined search query.
- `api/search/index.php:30-51` προσθέτει filters.
- `api/search/index.php:55-56` κάνει prepare και execute.
- `api/search/index.php:58-65` επιστρέφει filters και data σαν JSON.

Κώδικας να δείξεις:

```
api_response([
    'filters' => [
        'q' => $keyword,
        'status' => $status,
        'specialty_id' => $specialtyId,
    ],
],
```

```
'data' => $stmt->fetchAll(),  
]);
```

Τι να πεις:

Αυτό το endpoint δίνει την ίδια search δυνατότητα με την HTML σελίδα, αλλά επιστρέφει JSON για Postman ή browser testing.

Άλλα API endpoints:

- `api/stats/index.php:6-13` επιστρέφει counts για users, specialties, lists, candidates, entries και tracking.
- `api/specialties/index.php` επιστρέφει specialties.
- `api/lists/index.php` επιστρέφει appointment lists joined με specialties.
- `api/candidates/index.php` επιστρέφει candidate profiles.
- `api/entries/index.php` επιστρέφει ranked candidate entries.

Postman:

- `postman/CSE_326_Web_project.postman_collection.json`

Μετάβαση:

Τέλος, θα αναφέρω το UI και το responsive styling.

14. UI Και Styling

Άνοιξε: `includes/header.php`

Σημαντικές γραμμές:

- `includes/header.php:1-18` υπολογίζει το app base URL.
- `includes/header.php:111-116` φορτώνει Bootstrap και custom CSS.
- `includes/header.php:20-42` ορίζει shared footer rendering.
- `includes/header.php:45-81` ορίζει breadcrumbs.
- `includes/header.php:83-106` προσθέτει skip link, main wrapper, breadcrumbs και footer.

Άνοιξε: `assets/css/styles.css`

Σημαντικά:

- Είναι το custom design file.
- Δουλεύει μαζί με Bootstrap.
- Κάνει styling σε topbar, dashboards, cards, search results, forms, tables και responsive layouts.
- Το folder λέγεται `assets`, όχι `assets`.

Τι να πεις:

Το UI χρησιμοποιεί Bootstrap μαζί με custom CSS. Το shared header φορτώνει το ίδιο stylesheet παντού, οπότε homepage, login, dashboard, CRUD pages και API links φαίνονται σαν μία ενιαία εφαρμογή.

Μετάβαση:

Θα κλείσω με το checklist coverage.

15. Κλείσιμο / Checklist

Πες στο τέλος:

Συνοπτικά, το project καλύπτει τα βασικά requirements: έχει relational MySQL database με 7 tables, authentication με PHP sessions, δύο roles, protected member και admin pages, CRUD για τέσσερα admin entities, member tracking, searchable appointment-list page, JSON API endpoints, Postman collection και responsive styling.

Δείξε αν χρειαστεί:

- `README.md` για overview και install instructions.
- `database/schema.sql` για database structure.
- `auth/login.php` και `auth/register.php` για authentication.
- `modules/list.php` για search.
- `modules/admin/dashboard.php` για admin entry.
- `api/_bootstrap.php` και `api/search/index.php` για JSON API.

16. Αν Σε Ρωτήσουν Για Άδεια / Legacy Files

Μπορεί να έχεις ανοιχτά:

- `includes/nav.php`
- `modules/submit/dashboard.php`
- `submit/index.html`
- `admin/index.html`

Πες:

Αυτά είναι legacy ή placeholder files από προηγούμενη δομή του project. Η πραγματική τωρινή εφαρμογή χρησιμοποιεί `auth/`, `modules/`, `api/`, `includes/` και `database/`. Τα root `login.php`, `register.php` και `logout.php` είναι compatibility redirects προς τα νεότερα αρχεία μέσα στο `auth/`.

17. Χρονισμός Για 10 Λεπτά

```

0:00 - 0:45    Project summary και homepage
0:45 - 1:45    Database connection και helper functions
1:45 - 3:00    Register/login/session flow
3:00 - 4:00    User dashboard
4:00 - 5:15    Search page
5:15 - 6:00    Member tracking
6:00 - 7:15    Admin dashboard και CRUD pattern
7:15 - 8:15    Database schema και relationships
8:15 - 9:15    API και Postman
9:15 - 10:00   UI, checklist, closing summary

```

18. Καλύτερη Διαδρομή Για Browser Demo

```

1. Open /index.php
2. Log in as admin@example.com / Password123!
3. Show /modules/dashboard.php
4. Open /modules/list.php and filter search
5. Open /modules/tracking/dashboard.php
6. Open /modules/admin/dashboard.php
7. Open one CRUD page, preferably Specialties
8. Open /api/stats/index.php
9. Open /api/search/index.php?q=maria&status=active

```

19. Οι Πιο Σημαντικές Γραμμές Να Θυμάσαι

includes/db.php:30-38	PDO connection
includes/functions.php:15-20	require_auth()
includes/functions.php:42-50	require_admin()
includes/functions.php:52-55	e() output escaping
auth/register.php:60-68	insert user + password_hash()
auth/login.php:22-31	find user + password_verify() + session
modules/dashboard.php:7	protected dashboard
modules/list.php:30-33	search joins
modules/list.php:64-66	prepared search execution
modules/tracking/dashboard.php:30-41	tracking insert/update
modules/admin/dashboard.php:7	admin guard
modules/specialties/dashboard.php:37-58	CRUD insert/update pattern
database/schema.sql:48-65	candidate_list_entries table
api/_bootstrap.php:6-13	JSON API response helper
api/search/index.php:58-65	JSON search response